Frontend

Frontend Application Specification

This document describes the web application architecture, page routes, wallet integration layer, state management, and build setup for UnifyVault V2 (`apps/web-v2`).

—

1. Overview

The UnifyVault frontend is a server-side rendered (SSR) and client-side interactive Web3 application built using **Next.js 15 (App Router)**, **React 19**, **TypeScript**, **TailwindCSS**, **Wagmi v2**, **Viem v2**, **RainbowKit v2**, and **TanStack Query v5**.

—

2. Directory Structure

```
apps/web-v2/
├── app/
│   ├── page.tsx              # High-Conversion Landing Page & Protocol Showcase
│   ├── app-home/page.tsx     # Main Protocol Dashboard & Vault Metrics
│   ├── deposit/page.tsx      # Deposit Collateral Page (USDC / cbBTC / WETH)
│   ├── redeem/page.tsx       # Redeem Shares Page (Net Collateral Withdrawal)
│   ├── transfer/page.tsx     # Direct UVBE Share Transfer Page (with QR Code Camera Scanner)
│   ├── predict/page.tsx      # Flash 30s Rapid Binary Prediction Game (Custom Multipliers)
│   ├── staking/page.tsx      # UVBE Staking Vault & Rewards Engine
│   ├── portfolio/page.tsx    # User Holdings, Cost Basis, Realized/Unrealized P&L
│   ├── analytics/page.tsx    # Historical NAV & Performance Charts
│   ├── treasury/page.tsx     # Protocol Treasury & Fee Metrics
│   ├── contracts/page.tsx    # Deployed Contract Addresses & BaseScan Links
│   ├── transactions/page.tsx # Filterable User Transaction History
│   ├── p2p/page.tsx          # P2P Marketplace, Limit Orderbook, OCR & Escrow
│   ├── api/                  # Serverless Backend Endpoints
│   │   ├── p2p/              # UPI / Fiat Payment Intents, Claims, & Disputed Trades
│   │   ├── flashpulse/       # Flash 30s Game Vault Balance & Settle Logic
│   │   └── smart-account/    # Gasless Bundler & Sponsorship Relayer
│   └── admin/                # Admin & Keeper Management Console
│       ├── page.tsx          # Admin Overview
│       ├── custody/page.tsx  # Vault Custody Management
│       ├── oracle/page.tsx   # Oracle Price Feeds & Multi-State Staleness Monitor
│       ├── rebalance/page.tsx # Portfolio Rebalancing Triggers
│       ├── treasury/page.tsx  # Protocol Fee Sweeps
│       └── settings/page.tsx  # Rate Limits & Slippage Settings
├── components/               # React UI Components (Vault, P2P, Common Modals, QrScannerModal)
├── constants/                # Contract ABIs, Addresses, & Chain Configs
├── hooks/                    # Custom React Web3 Hooks (useVault, useOracle, useP2PEscrow,
useFlashPulse, useSmartAccount)
├── lib/                      # Formatting, Math, Payment Store, & Portfolio Transforms
└── providers/                # Web3 & Query Providers (Wagmi, RainbowKit, TanStack)
```

—

3. Application Routes & Pages

- **Route**: Page Title | Key Capabilities
- `/`: Landing Page | Institutional hero, live TVL/NAV metrics ticker, strategy breakdown, and Web3 connection CTA.

- **`/app-home`**: App Dashboard | Protocol TVL, NAV per share, active price ticker, target allocation, and quick deposit/redeem actions.
- **`/deposit`**: Deposit | Multi-asset deposit (USDC, cbBTC, WETH), slippage tolerance, gasless 1-click batching for Smart Accounts, and share preview.
- **`/redeem`**: Redeem | Burn `UVBE` shares for net collateral with customizable slippage protection and deadline guards.
- **`/transfer`**: Transfer | Direct wallet-to-wallet transfer of `UVBE` share tokens with real-time camera QR scanner and proportional cost basis preservation.
- **`/predict`**: Flash 30s | 30-second rapid prediction game with Pyth Oracle live pricing, gasless game vault, Parimutuel odds, and customizable reward multipliers (2x to 20x).
- **`/staking`**: Staking Vault | UVBE permanent & flexible staking pools with referral commissions and yield claim engine.
- **`/portfolio`**: Portfolio | Personal share holdings, on-chain cost basis (`CostBasisManagerV2`), entry prices, realized P&L, and token breakdown (Grid & Table views).
- **`/analytics`**: Analytics | Interactive charts of historical NAV, daily volume, fee accumulation, and asset price performance.
- **`/p2p`**: P2P Marketplace | Non-custodial limit orderbook, counter-order matching, trade escrow (`P2PEscrowV2`), QR code generation, OCR receipt upload, and dispute arbitration.
- **`/treasury`**: Treasury | Protocol fee reserve monitoring, collateral reserve breakdowns, and high-water mark fee logs.
- **`/contracts`**: Contracts | Verifiable directory of all deployed Base Sepolia & Base Mainnet smart contracts with direct BaseScan explorer links.
- **`/transactions`**: Transactions | Filterable transaction ledger covering deposits, redemptions, transfers, and P2P escrow lifecycle events.
- **`/admin/*`**: Admin Console | Role-gated management console for emergency pausing, oracle monitoring, rate limit adjustments, custody management, and rebalancing.
- **`/admin/oracle`**: Oracle Manager | Live feed monitoring with explicit multi-state staleness badges (`LIVE`, `STALE`, `REVERTED`, `UNAVAILABLE`) and atomic refresh.

—

4. ERC-4337 Account Abstraction & Gasless UX

- **1-Click Atomic Batching**:
- **Deposit**: `[USDC.approve(Controller, amount), Controller.deposit(USDC, amount, minShares, receiver)]` in a single transaction.
- **P2P Escrow**: `[UVBE.approve(P2PEscrow, amount), P2PEscrow.fundTrade(tradeId)]` without dual wallet confirmation prompts.
- **Paymaster Policy**:
- Validates UserOperations against strict security whitelist in `paymasterPolicy.ts`.
- Rejects native ETH draining, unapproved spenders, and unwhitelisted targets.

—

5. Phase F5 Runtime Performance & UI Integrity

- **React Memoization & Query Caching**: High-frequency RPC reads use TanStack Query with structured stale-times (30s–60s) to eliminate duplicate RPC calls and re-render loops.
- **Zero Horizontal Overflow**: Fully tested and verified across 11 standard viewports (320px to 1920px) with 0px horizontal overflow.
- **Data-Width & Layout Integrity**: Token balances use `formatDisplayCryptoBalance` to cleanly render long 18-decimal balances without column collisions, while preserving full unrounded on-chain values in hover tooltips.

—

6. Oracle Feed Freshness & Multi-State Error Handling

The application implements a robust, transparent oracle pricing pipeline:

1. **Multi-State Feed Classification**:

- `LIVE`: Price is valid, non-zero, and timestamp is within heartbeat limits (`isPriceFresh == true`).
- `STALE`: Heartbeat has expired or feed is delayed.
- `REVERTED`: Underlying provider reverted (e.g. circuit breaker tripped or feed paused).
- `UNAVAILABLE`: Unconfigured or uninitialized feed.

1. **Graceful UI Fallbacks**:

- Non-live feeds render explicit `‘Price unavailable’` and `‘Value unavailable’` status labels rather than falsely coercing errors to `$0.00`.

1. **Atomic Multi-Query Refresh**:

- The `/admin/oracle` “Refresh Feeds” handler atomically invalidates and refetches all oracle queries, `PortfolioManager.calculateUVPrice()`, vault TVL, and portfolio metrics concurrently via `Promise.allSettled`.

—

5. Web3 & Wallet Integration

- **Wallet Connection**: Integrated via `@rainbow-me/rainbowkit` and `wagmi`. Supports MetaMask, Coinbase Wallet, WalletConnect v2, and browser extension wallets.
- **Chain Support**: Configured for **Base Mainnet** (`chainId: 8453`) and **Base Sepolia** (`chainId: 84532`).
- **RPC Communication**: Direct EVM JSON-RPC calls via `viem` public clients. Contract addresses are resolved dynamically from `ProtocolDirectory`.

—

6. Build Process

To build the frontend for production:

```
# From workspace root
pnpm --filter @unifyvault/web-v2 build

# Run locally in production mode
cd apps/web-v2 && pnpm start -p 3001
```